Take-home Challenge: Document Approval Workflow

Setup Instructions

To submit your work, please follow the steps below:

1. **Code Repository**
   Create a private Git repository on a platform like GitHub or Bitbucket. Share access with enghiring@springboard.com. Ensure your development progress is reflected through meaningful commits.
2. **Recommended Tech Stack**
   a. **Backend**: Django
   b. **Frontend**: React

   These are our recommendations — you're free to use any other stack you're comfortable with.
3. **Video Walkthrough**
   Install and use Loom (or a similar screen recording tool) on your desktop. Record a brief video demoing the functionality you've built and explaining your approach. *Recommended length: 5 minutes.*

Overview

Build a minimal Document Approval Workflow with a backend API and a frontend client. The application should let a user submit a document for review, and let a reviewer approve or reject it with a status that the submitter can track. You are expected to implement the specified feature set within the time budget below. This is the first iteration — we'll build on top of this in subsequent interview rounds.

You're encouraged to focus on clean, modular design, reasonable edge case handling, and test coverage.

What We're Evaluating

We're evaluating your ability to:

- Design a clean, extensible system end-to-end
- Implement functionality in a modular and testable way
- Handle edge cases and errors gracefully
- Write well-structured, readable code
- Reason through product requirements and translate them into technical solutions

Feature Requirements

Functionality

A submitter should be able to:

- Submit a document (a file, or an input) tied to their user
- View the status of their submission (pending, approved, or rejected)

A reviewer should be able to:

- View a list of submitted documents
- Approve or reject a submission

Back-End

- Provide APIs that enable:
  - Creating a document submission
  - Listing/fetching submissions and their current status
  - Updating a submission's status (approve/reject)
- Input validation: handle invalid payloads with appropriate error responses
- Use storage of your choice (database, in memory, file-based, etc.)

Front-End

- Create a web app using React or React-Based Frameworks.
- A view for submitting a document and seeing its status
- A view for a reviewer to see pending submissions and approve/reject them
- Handle error states

Bonus

- Hosting the app on a public URL (e.g., Vercel, Netlify, Heroku, AWS) — share the live link in your README
- Additional UI/UX polish (e.g., loading states, filtering submissions by status)

> If not hosted, ensure your README has clear setup and run instructions so we can run it locally.

Deliverables

- **Source Code**: The private repository you set up in Setup Instructions above.
- **Video Walkthrough**: The ~5 minute video described in Setup Instructions above — demoing what you built and explaining your approach. Include the link in your README.
- **README**: We expect a detailed README covering:

    - Setup instructions (for both FE and BE)
    - **API design**: endpoints, request/response shapes, and the reasoning behind them. A Swagger spec is a good-to-have here, but not required.
    - **Major decisions made**: your data model, status/state transitions, and any trade-offs you made given the time budget
    - **AI tools used**: which tool(s), and roughly how you used them (e.g. scaffolding, debugging, writing tests)
    - What you'd add for a production-ready environment
- **Tests**:
    - Cover critical functionality with unit or integration tests
    - Aim for a meaningful test suite that covers both happy and edge cases
- **Test Execution**: Provide a single command or script to run all tests

> ℹ No strict line coverage requirement, but we expect coverage of core logic and critical flows.

Time Budget

- You are expected to spend **up to 2 hours** on this challenge.
- You're free to mock parts of the system (like data or file storage) to stay within this limit.

Final Notes

- Avoid sharing hardcoded API schemas; instead, design interfaces that suit your architecture.
- Structure your code into logical modules.
- Don't over-optimize. We want to see clean, working code that is easy to build upon.
- Use whichever AI tool you want for building. Just add a note in your README on which tool(s) you used and how, and be ready to walk us through what you used it for in the video and subsequent rounds.

We're excited to see your implementation. Good luck, and thank you for taking the time!